

Monte Carlo Equity Estimation and Expected-Value Decision Policy

for an Autonomous Heads-Up Limit Texas Hold'em Agent

Capstone Report

July 22, 2026

Abstract

This report documents the design and mathematical justification of an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE), built entirely from first principles without pre-built poker libraries or downloaded strategy tables. The agent's policy rests on two components: (1) a Monte Carlo rollout procedure that estimates hand equity against an unknown opponent hand and unseen community cards, and (2) a closed-form Expected Value (EV) decision rule that converts that equity estimate into a fold/call/raise action using pot odds. We derive the EV decision boundary, justify the Monte Carlo sampling procedure with a variance argument, and report empirical performance against three baseline opponents.

1 Problem Setting

Heads-Up Limit Hold'em is a game of *imperfect information*: at every decision point the acting player observes only their own two hole cards and the public community cards, while the opponent's hole cards and the undealt remainder of the deck are hidden random variables. This is the central difference from the fully-observed CartPole control problem that preceded this capstone – there, the state was Markov and complete; here, the true game state (both hands plus the deck order) is only partially revealed, and any decision policy must reason under uncertainty rather than against a known state.

We address this by treating the hidden information as a random variable to be integrated out: rather than trying to infer the opponent's exact hand, we estimate the *distribution* of outcomes by sampling plausible completions of the unknown information and averaging.

Chosen archetype. This agent does *not* use Reinforcement Learning, so reward shaping is not applicable here. Instead it uses a **mathematical heuristics architecture**: a Monte Carlo estimator converts hidden information into a single scalar (equity), and a closed-form Expected Value calculation converts that scalar directly into an action, with no learned parameters, no training loop, and no reward signal to shape. Sections 2–4 justify each stage of that pipeline in turn.

2 State Representation

At each decision point the agent receives exactly the tuple specified by the arena API:

(hole_cards, community_cards, pot_size, stack_size, amount_to_call, legal_actions)

We deliberately do *not* hand-craft a reduced feature vector (e.g. Chen score, bucketed preflop tiers) from these inputs. Instead, the raw card identities are consumed directly by the hand evaluator inside the Monte Carlo loop described in Section 3, and the four numeric quantities (`pot_size`, `stack_size`, `amount_to_call`, and the implied `pot_odds`) feed directly into the EV decision rule in Section 4. This keeps the state representation exact – no information is thrown away by discretization – at the cost of pushing the computational burden onto the equity estimator, which is the tradeoff this report defends.

2.1 Hand Evaluation

Given up to 7 known cards (2 hole + up to 5 community), the agent’s best achievable 5-card hand is found by generating every 5-card subset with `itertools.combinations` and scoring each with a comparable tuple

$$\text{score} = (\text{category}, \text{tiebreak}_1, \text{tiebreak}_2, \dots)$$

where `category` $\in \{0, \dots, 8\}$ ranks High Card through Straight Flush, and the tiebreak entries are the relevant card ranks in descending priority (e.g. trip rank then pair rank for a Full House). Because Python compares tuples lexicographically, the best hand over all $\binom{7}{5} = 21$ subsets is simply

$$\text{best_score} = \max_{c \in \binom{7 \text{ cards}}{5}} \text{score}(c),$$

with rank-count grouping done via `collections.Counter`. This gives an exact evaluator with no external library dependency, verified against known hand categories (royal flush, quads, full house, the wheel straight A-2-3-4-5, and boardplay/tie scenarios) before use in the agent.

3 Monte Carlo Equity Estimation

3.1 Definition

We define *equity* as the probability of winning the pot at showdown, counting a split pot as half a win:

$$\text{Equity} = P(\text{win}) + \frac{1}{2}P(\text{tie}).$$

Because the opponent’s two hole cards and any undealt community cards are unobserved, equity cannot be computed in closed form without enumerating every possible completion of hidden information (exact enumeration is tractable on the river, where only the opponent’s $\binom{45}{2} = 990$ hands are unknown, but becomes expensive earlier when several board cards remain undealt as well). We instead estimate it by Monte Carlo simulation.

3.2 Procedure

Algorithm 1 Monte Carlo Equity Estimation

```

1: function ESTIMATEEQUITY(hole, community,  $N$ )
2:   known  $\leftarrow$  hole  $\cup$  community
3:   deck  $\leftarrow$  (52-card deck)  $\setminus$  known
4:    $k \leftarrow 5 - |\text{community}|$   $\triangleright$  cards still to come
5:   total  $\leftarrow 0$ 
6:   for  $i = 1$  to  $N$  do
7:     sample  $(2 + k)$  cards uniformly without replacement from deck
8:     opp_hole  $\leftarrow$  first 2 sampled cards
9:     rest_of_board  $\leftarrow$  remaining  $k$  sampled cards
10:    full_board  $\leftarrow$  community  $\cup$  rest_of_board
11:    my_score  $\leftarrow$  BestHandScore(hole  $\cup$  full_board)
12:    opp_score  $\leftarrow$  BestHandScore(opp_hole  $\cup$  full_board)
13:    if my_score  $>$  opp_score then total  $\leftarrow$  total +1
14:    else if my_score = opp_score then total  $\leftarrow$  total +0.5
15:    end if
16:  end for
17:  return total /  $N$ 
18: end function

```

Each trial samples *both* the opponent’s hand and the rest of the board jointly and consistently from the same reduced deck, so the two sources of hidden information (opponent cards, future community cards) are handled uniformly by a single sampling step rather than two separate procedures.

3.3 Convergence and Variance

Let $\hat{p}_N$ be the Monte Carlo equity estimate after N trials, and let p be the true equity. Each trial is (approximately¹) a Bernoulli-like observation with mean p , so by the Law of Large Numbers $\hat{p}_N \rightarrow p$ as $N \rightarrow \infty$, and by the Central Limit Theorem the standard error is

$$\text{SE}(\hat{p}_N) \approx \sqrt{\frac{p(1-p)}{N}}.$$

At the worst case $p = 0.5$, this gives $\text{SE} \approx \frac{0.5}{\sqrt{N}}$. Table 1 shows the resulting standard error at the trial counts we evaluated.

N (trials)	SE at $p = 0.5$	Approx. wall-time per decision
80	$\pm 5.6\%$	≈ 22 ms
200	$\pm 3.5\%$	≈ 59 ms

Table 1: Standard error and measured latency vs. trial count, benchmarked on a peflop equity query.

¹Trials are drawn without replacement from a finite deck, so they are not exactly i.i.d.; however for a 52-card deck with at most 7 cards removed, the dependence between trials is small and the i.i.d. approximation is standard practice in poker equity calculators.

We selected $N = 80$ as the deployed trial count. This keeps decision latency low enough that a 10,000-hand tournament (at most 4 decisions per hand) completes in minutes rather than tens of minutes, while keeping the equity estimate’s standard error under 6% – small relative to the $\sim 12\%$ gap between our raise threshold (0.62) and a coin-flip hand (0.50), so sampling noise rarely flips a decision near the threshold. N is exposed as a single named constant (`MC_TRIALS`) so it can be re-tuned against the arena’s actual time budget without touching decision logic.

4 Expected Value Decision Rule

4.1 Derivation

Consider the decision to call a bet of size `amount_to_call` = c when the pot (before the call) is P . If we call, we win $P + c$ with probability p (our estimated equity) and lose c with probability $1 - p$:²

$$\text{EV}(\text{call}) = p \cdot (P + c) - (1 - p) \cdot c.$$

Expanding and solving for the break-even equity:

$$\text{EV}(\text{call}) = 0 \iff p(P + c) = (1 - p)c \iff p = \frac{c}{P + c}.$$

This threshold is exactly the standard *pot odds* quantity:

$$\text{pot_odds} = \frac{\text{amount_to_call}}{\text{pot_size} + \text{amount_to_call}}.$$

So the mathematically justified rule is:

$$\text{Call is +EV} \iff \hat{p} > \text{pot_odds}, \quad \text{Call is -EV} \iff \hat{p} < \text{pot_odds}.$$

4.2 Policy

The full policy layers a value-betting threshold on top of the fold/call boundary above:

$$\text{action} = \begin{cases} \text{FOLD} & \text{if } c > 0 \text{ and } \hat{p} < \text{pot_odds}, \text{ and FOLD is legal} \\ \text{RAISE} & \text{if } \hat{p} > \tau_{\text{raise}}, \text{ and RAISE is legal} \\ \text{CALL} & \text{otherwise (includes checking when } c = 0) \end{cases}$$

where $\tau_{\text{raise}} = 0.62$. The fold branch is the EV break-even condition derived above; the raise branch is a value-betting heuristic rather than a derived quantity – we raise when our equity is comfortably ahead of a coin flip, since a raise both grows the pot while we are likely ahead *and* folds out some fraction of weaker opponent hands, both of which are positive-EV effects that our single-street pot-odds formula above does not capture (see Section 6).

4.3 Fold-Safety Behavior

The engine’s action space always includes `CALL` at $c = 0$ (a check), so the agent has no scenario in which all branches are unavailable; a final defensive fallback returns the first legal action if reached, purely to guarantee the arena never sees an invalid return value from a bug.

²This treats the hand as ending immediately after the call, i.e. it ignores implied odds from future betting rounds. See Section 6 for discussion.

5 Empirical Results

The agent was tested locally (200 hands per matchup, alternating the dealer button each hand since stacks reset every hand per the game spec) against three baseline opponents built for this purpose:

- **RandomBot**: selects uniformly among legal actions.
- **CallingStationBot**: always calls/checks, never folds or raises.
- **TightAggressiveBot**: a no-rollout heuristic that raises with pairs or high cards and folds weak hands to a bet – a cheap proxy for a competent-but-simple opponent.

Opponent	Hero result (200 hands)	BB/100
RandomBot	+394 chips	+98.5
CallingStationBot	+313 chips	+78.2
TightAggressiveBot	+169 chips	+42.2

Table 2: Chip-delta results of CustomPokerBot ($N = 80$ trials) versus baseline opponents.

The ordering of win rates is consistent with expectations: the agent exploits CallingStationBot’s inability to fold to negative-equity spots and RandomBot’s lack of any pot-odds awareness most strongly, and beats the more disciplined TightAggressiveBot by a smaller but still clearly positive margin, since that opponent occasionally makes reasonable folds/raises on its own. No invalid actions or exceptions occurred over the full test batch, confirming the engine’s legality guarantees (Section 4.3) and the evaluator were exercised correctly across thousands of individual betting decisions.

6 Limitations and Future Work

1. **No implied odds.** The EV derivation in Section 4.1 treats the hand as ending after the current call, ignoring future betting on later streets. A more complete model would discount pot odds by the probability-weighted value of future action.
2. **No opponent modeling.** Equity is estimated against a uniformly random opponent hand rather than a range conditioned on the observed betting so far. A fixed raise threshold is therefore exploitable by an opponent who folds too often to aggression, or who never bluff-raises.
3. **No mixed strategies.** The policy is deterministic given $(\hat{p}, \text{pot_odds})$, so it has no bluffing frequency and is theoretically exploitable by game-theory-optimal (GTO) opponents, which by definition randomize their actions at the equilibrium indifference points. Extending toward Counterfactual Regret Minimization (CFR) over an abstracted betting tree, as outlined in the original assignment brief, would address this and is the natural next step.
4. **Sampling variance near the threshold.** As shown in Table 1, $N = 80$ trials carries roughly $\pm 5\text{--}6\%$ standard error, so hands whose true equity sits within a few points of pot_odds or τ_{raise} are decided somewhat noisily. Increasing N trades tournament wall-clock time for lower variance.

7 Conclusion

The agent combines an exact, from-scratch 7-card hand evaluator with a Monte Carlo equity estimator and a closed-form EV decision boundary derived directly from the pot-odds identity $p = c/(P+c)$. This keeps the strategy mathematically transparent – every fold/call decision reduces to a single inequality with a derivable threshold – while the Monte Carlo layer lets the agent handle the imperfect-information aspects of the game (hidden opponent cards, undealt community cards) without any hand-tuned lookup table. Empirical testing against three baseline opponents shows consistent positive win rates with no engine or evaluator failures, validating both the mathematical approach and its implementation.